



RV College of Engineering

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

MAJOR PROJECT REPORT (CS481P)

Submitted by,

Nikhil Vasu

1RV22CS128

Rohith Biradar

1RV22CS164

Suraj Chanaveeragoudra

1RV22CS211

Under the guidance of

Dr. Nagaraja G.S

Sanjana Ravindra Otihal

Professor

Assistant Professor

Dept. of CSE

Dept. of CSE

RV College of Engineering

RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering

in

Computer Science and Engineering

Department of CSE

2025-26





RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

A Major Project Report (CS481P)

Submitted by,

Nikhil Vasu

1RV22CS128

Rohith Biradar

1RV22CS164

Suraj Chanaveeragoudra

1RV22CS211

Under the guidance of

Dr. Nagaraja G.S

Sanjana Ravindra Otihal

Professor

Assistant Professor

Dept. of CSE

Dept. of CSE

RV College of Engineering

RV College of Engineering

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering®, Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)
Department of Computer Science and Engineering



CERTIFICATE

Certified that the major project (CS481P) work titled ***Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection*** is carried out by **Nikhil Vasu (1RV22CS128)**, **Rohith Biradar (1RV22CS164)** and **Suraj Chanaveeragoudra (1RV22CS211)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	Head of the Department	Dean CSE Cluster	Principal
Dr. Nagaraja G.S	Dr. Shanta Rangaswamy	Dr. Ramakanth Kumar P	Dr. K. N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **Nikhil Vasu, Rohith Biradar** and **Suraj Chanaveeragoudra** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Nikhil Vasu(1RV22CS128)
2. Rohith Biradar(1RV22CS164)
3. Suraj Chanaveeragoudra(1RV22CS211)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Nagaraja G.S**, Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel members **Dr. Nagaraja G.S**, Professor, Department of CSE, RVCE and **Sanjana Ravindra Otihal**, Assistant Professor, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Dr. Chethana R Murthy**, Professor, Department of CSE and **Prof. Deepika Dash**, Assistant Professor, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Trade-Based Money Laundering (TBML) has emerged as a significant challenge within modern financial systems due to the increasing complexity and scale of global trade transactions. Conventional Anti-Money Laundering (AML) systems primarily rely on isolated rule-based monitoring frameworks that are often ineffective in identifying hidden transactional dependencies and multi-hop laundering typologies across interconnected banking networks. To address these limitations, this work developed a privacy-preserving Federated Heterogeneous Graph Neural Network (HGNN)-based framework for real-time multi-class TBML detection across distributed institutional environments.

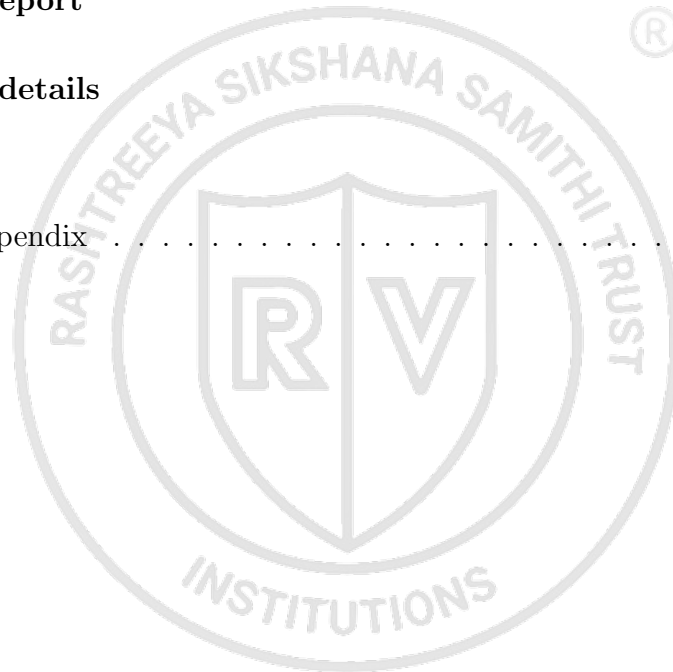
The proposed framework modeled accounts, trade entities, SWIFT transactions, and trade documents as heterogeneous financial graphs using Memgraph. A hybrid analytical architecture integrating Graph Attention Network (GAT)-based convolutional layers and Long Short-Term Memory (LSTM)-based temporal sequence modeling was employed to capture relational topology and evolving transactional behavior. Federated learning using the Federated Averaging (FedAvg) aggregation strategy enabled collaborative cross-bank model optimization without centralized transactional data sharing. The implemented workflow incorporated Apache Kafka-based transaction streaming, graph construction, federated fraud inference, Suspicious Transaction Report (STR) generation, and regulator-facing compliance synchronization using Python, PyTorch Geometric, FastAPI, PostgreSQL, Memgraph, and React.js.

Experimental evaluation across distributed banking clients demonstrated stable federated convergence with Fraud-class Precision values approaching 0.98, F1-Scores nearing 0.84, and Precision-Recall Area Under Curve (PR-AUC) values exceeding 0.92. Concurrent scalability evaluation additionally maintained dashboard retrieval latency near 156–162 ms and fraud-filtering response latency near 148–159 ms under increasing institutional workloads, thereby validating the operational suitability of the proposed framework for scalable and privacy-preserving TBML detection. The framework can be extended into a fully integrated banking compliance platform with automated risk response, transaction control, and regulatory workflows. These enhancements would transform it from a monitoring system into a complete financial crime prevention ecosystem.

CONTENTS

Abstract	i
List of Figures	iv
List of Tables	v
List of Abbreviations	vi
List of Symbols	vii
1 Introduction to Trade-Based Money Laundering	1
1.1 Overview of Trade-Based Money Laundering	2
1.1.1 Global Financial Crime Scenario	2
1.1.2 Trade-Based Money Laundering	3
1.1.3 Problem Area	3
1.2 Literature Review	4
1.3 Motivation	7
1.4 Problem Statement	7
1.5 Objectives	7
1.6 Brief Methodology of the project	8
1.7 Assumptions and Constraints of the Work	10
1.8 Organization of the Report	11
2 Theory and Fundamentals of Project	12
2.1 Trade-Based Money Laundering	13
2.1.1 Introduction to TBML	13
2.1.2 Common Techniques Used in TBML	14
2.1.3 Challenges in Detecting TBML	15
2.2 Algorithms Used	16
2.2.1 Graph Attention Network v2 (GATv2)	16
2.2.2 Long Short-Term Memory (LSTM)	20
2.3 Federated Learning	23

2.3.1	Introduction	24
2.3.2	Working of Federated Learning	25
2.3.3	Advantages of Federated Learning	26
2.3.4	Limitations of Federated Learning	26
2.4	User Interface	27
2.4.1	Modern Web Application Frameworks	27
2.4.2	Component Lifecycle and Reactivity	28
2.4.3	Data Visualization	29
2.5	Summary	30
A	Plagiarism report	31
B	Publication details	33
C	Code	35
C.1	First Appendix	36
	Bibliography	37



LIST OF FIGURES

1.1	Proposed Methodology for TBML Detection	9
2.1	Trade-Based Money Laundering through trade misinvoicing and open-account transactions	14
2.2	GATv2 Architecture and Working	17
2.3	Computing the input gate, forget gate, and output gate in an LSTM model	21
2.4	Computing the input node in an LSTM model	22
2.5	Computing the memory cell internal state in an LSTM model	22
2.6	Computing the hidden state in an LSTM model	23
2.7	General Outlook of Federated Learning	24
2.8	Federated Learning General Process in Central Orchestrator Setup	25
2.9	Federated Learning Workflows showing (a) FL-Aggregation Server, (b) FL-Peer-to-Peer, and (c) Centralized Training architectures.	26

LIST OF TABLES

1.1	Key Statistics Related to Global Trade-Based Financial Crime	3
2.1	Comparison Between GAT and GATv2	20



ABBREVIATIONS

AI Artificial Intelligence

AML Anti-Money Laundering

ERC Electrical Rule Check

FedAvg Federated Averaging

FL Federated Learning

GAT Graph Attention Network

GCN Graph Convolutional Network

GNN Graph Neural Network

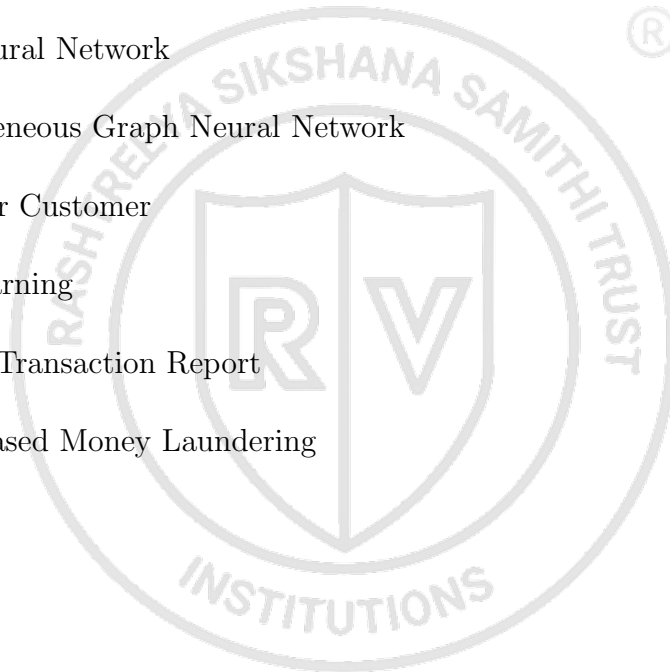
HGNN Heterogeneous Graph Neural Network

KYC Know Your Customer

ML Machine Learning

STR Suspicious Transaction Report

TBML Trade-Based Money Laundering



LIST OF SYMBOLS

λ Wavelength of a light in a medium

τ Time constant

c Speed of light

f Frequency of light in a medium



Chapter 1

**Introduction to Trade-Based Money
Laundering**

CHAPTER 1

INTRODUCTION TO TRADE-BASED MONEY LAUNDERING

International trade and digital finance have made global transactions faster, but also harder to monitor. Criminals increasingly exploit this complexity to move illicit value through legitimate trade activity, exposing the limits of conventional Anti-Money Laundering (AML) systems. Trade-Based Money Laundering (TBML) is especially difficult to detect because the suspicious activity is hidden inside normal commercial exchanges. This creates a clear need for intelligent, scalable, and privacy-preserving methods that can uncover hidden transactional relationships in real time.

“For financial institutions, TBML is uniquely challenging because illicit funds are embedded within legitimate trade transactions, leaving traditional AML controls blind to the risks.”

– Dr. Sebastian Hetzler, IMTF Co-CEO [1]

TBML works by blending illicit financial flows into ordinary trade transactions, which reduces the effectiveness of rule-based compliance checks and isolated transaction monitoring.

1.1 Overview of Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) refers to disguising the proceeds of crime through legitimate international trade transactions [2]. Instead of moving money directly, TBML hides illegal value inside import-export activities, invoice settlements, and cross-border trade records. Because these transactions involve multiple intermediaries, institutions, and jurisdictions, conventional AML systems struggle to identify suspicious patterns reliably.

1.1.1 Global Financial Crime Scenario

The growth of cross-border trade has made illicit activity harder to spot in normal financial flows. The United Nations Office on Drugs and Crime (UNODC) estimates that money laundering accounts for nearly 2% to 5% of global GDP, or about \$800 billion to \$2 trillion annually [3]. Europol also reports that authorities intercept less than 1% of

illicit financial flows [4]. These figures show how quickly laundering networks are evolving and why traditional monitoring tools are no longer enough.

As goods, services, and financial assets move across borders, criminal actors can hide suspicious transactions inside legitimate trade activity. Even as banks strengthen Know-Your-Customer (KYC) controls and transaction surveillance, TBML continues to exploit gaps in conventional monitoring.

Table 1.1: Key Statistics Related to Global Trade-Based Financial Crime

Financial Crime Metric	Estimated Value
Global GDP associated with money laundering annually	2% – 5%
Estimated annual TBML economic volume	\$1.6T – \$2.0T
Illicit flows linked to trade misinvoicing in developing economies	87%
Global interception rate of illicit financial flows	<1%
Reduction in false positive alerts using AI-driven monitoring systems	Up to 40%

1.1.2 Trade-Based Money Laundering

TBML allows criminal organizations to move illicit value through trade while avoiding direct cash movement. Common techniques include over-invoicing, under-invoicing, multiple invoicing, false goods descriptions, phantom shipments, and shell entities that distort trade records. Because these actions are spread across real trade operations, they are difficult for regulators and financial institutions to detect.

Global Financial Integrity reports that trade misinvoicing and related TBML activity account for nearly 87% of illicit financial flows from developing economies [5]. Industry estimates also suggest that about \$1.6 trillion to \$2 trillion in illicit value is hidden each year within trade invoices and commercial documents [1]. Together, these findings show how heavily organized crime depends on the international trade system.

1.1.3 Problem Area

Conventional AML systems mainly rely on predefined rules, threshold-based monitoring, and manual investigations. Although these approaches can identify isolated suspicious transactions, they often fail to capture the interconnected relationships involved in coordinated money laundering activities. TBML networks typically involve importers,

exporters, shell companies, offshore entities, logistics providers, and financial intermediaries operating across multiple jurisdictions, making suspicious patterns difficult to trace using traditional approaches. Existing machine learning methods also face limitations because they generally analyze transactions as independent events, often overlooking hidden structural and relational patterns within financial networks.

As a result, traditional systems frequently generate high false positives, increase investigation delays, and reduce overall detection efficiency. In addition, privacy and regulatory restrictions prevent institutions from freely sharing sensitive transactional data, making centralized fraud detection impractical. These challenges highlight the need for scalable and privacy-preserving analytical frameworks capable of modeling complex distributed financial ecosystems while effectively detecting sophisticated TBML activities.

1.2 Literature Review

The growth of global finance has led to a large body of AML and TBML research. Earlier work focused on rule-based monitoring, but newer studies increasingly use machine learning, graph analytics, and distributed learning to detect hidden patterns in financial data. This shift is important because TBML does not usually appear as a single suspicious transfer; instead, it is spread across invoices, counterparties, shipping records, account movements, and commercial relationships. For that reason, recent literature has moved from isolated transaction screening toward methods that can model interactions between entities and uncover structural patterns in the wider financial network.

Jiani Fan, Lwin Khin Shar, Ruichen Zhang, Ziyao Liu, Wenzhuo Yang, Dusit Niyato, and Kwok-Yan Lam [6] surveyed deep learning methods for AML and showed that CNNs, RNNs, and GNNs can improve anomaly detection. Their survey is useful because it presents how learning-based techniques can extract richer patterns than fixed rules alone, especially in environments with large transaction volumes. At the same time, the work remained centered on centralized data processing, which makes it less suitable for institutions that cannot share sensitive records directly. This limitation is especially relevant in TBML scenarios, where data from multiple organizations may be needed to understand the full laundering chain.

Rasmus Ingemann Tuffveson Jensen and Alexandros Iosifidis [7] evaluated standard machine learning models for financial anomaly detection, including Logistic Regression, Random Forests, and Gradient Boosting methods. Their results showed that traditional

supervised models can outperform static rule-based approaches when the data is well prepared and labeled. However, the study still treated transactions largely as independent events. In practice, laundering activity is often distributed across connected accounts, counterparties, and repeated trade interactions, so a purely transaction-level view can miss the broader relational structure.

Jie Song, Sijia Zhang, Junghoon Park, Yu Gu, and Ge Yu [8] proposed a dimension expansion framework for learning hidden money laundering activities within transaction networks. The method strengthened transaction representation by combining neighboring account information with contextual interaction features, which is a useful step toward graph-aware fraud detection. The study showed that incorporating relational information can improve the detection of concealed laundering behavior across financial networks. Even so, the approach still relied on manually engineered features, which can reduce flexibility when the structure of the network changes or when the system must operate across distributed institutions.

In another study, Jie Song, Sijia Zhang, Pengyi Zhang, Junghoon Park, Yu Gu, and Ge Yu [9] introduced a Social Attention Graph Neural Network framework for detecting illicit accounts in blockchain transaction systems. Their work demonstrated the value of attention mechanisms in highlighting the most informative neighbors during graph learning. It also reinforced the idea that graph neural networks can outperform traditional machine learning models in suspicious-account detection. However, the study was focused on blockchain transactions rather than trade documents or invoice-based laundering, so it does not fully address the heterogeneous relationships that appear in TBML cases.

Zhong Li, Xueting Yang, and Changjun Jiang [10] proposed a Multi-View Graph-Based Hierarchical Representation Learning framework for organized money laundering group detection. Their use of multiple graph views helped identify coordinated laundering communities and showed that hierarchical graph representations can capture more than simple pairwise links. The work is valuable because it moves beyond flat feature vectors and models fraud as a collaborative activity. Still, the approach depended on predefined graph relations and centralized repositories, which can limit use in privacy-sensitive environments where institutions must keep their own data local.

Abdul Haluk Batur Gezici and Emre Sefer [11] proposed AMLPD, an unsupervised Anti-Money Laundering detection framework based on Personalized PageRank and graph

diffusion. This work is important because it reduces dependence on labeled data, which is often expensive and incomplete in real banking systems. The method was able to capture hidden graph structure in sparse financial networks and demonstrated that unsupervised learning can be practical for anomaly detection. However, the lack of supervised optimization also led to more false positives, which can increase the workload of compliance teams and reduce trust in the system.

Md. Rezaul Karim, Felix Hermsen, Sisay Adugna Chala, Paola De Perthuis, and Avikarsha Mandal [12] investigated scalable semi-supervised graph learning architectures for large-scale financial transaction monitoring. Their evaluation of SkipGCN and EvolveGCN showed that graph learning can scale to bigger transaction datasets while still preserving useful classification performance. This makes the work relevant for real-world fraud monitoring systems that must handle large volumes of records. Even so, the framework assumed centralized graph access and did not directly solve the privacy and regulatory challenges that arise when multiple institutions need to collaborate.

Kyungmo Koo, Minyoung Park, and Byungun Yoon [13] introduced an autoencoder-based suspicious transaction detection framework using Risk-Based Approaches (RBA) for banking systems. Their work demonstrated how compact latent representations of transaction data can simplify fraud detection while supporting internal banking surveillance. The study presents a computationally efficient alternative to complex graph-based models. However, the approach mainly focused on detecting isolated account-level anomalies and did not address the interconnected laundering relationships commonly observed in large-scale trade ecosystems.

Overall, recent research shows a gradual transition from traditional rule-based AML systems toward graph-driven and machine learning-based approaches. Despite these advancements, several challenges still remain in areas such as heterogeneous graph modeling, real-time fraud analysis, privacy-preserving collaboration, and scalability in distributed financial environments. In addition, many existing approaches are designed for centralized banking or cryptocurrency datasets and do not fully capture the complex trade relationships associated with TBML. These limitations motivate the need for a unified framework that combines heterogeneous graph learning, federated intelligence, and real-time detection capabilities.

1.3 Motivation

The work, “*Federated Heterogeneous Graph Neural Networks for Real-Time Detection of Trade-Based Money Laundering*”, was selected to address the growing complexity of financial crime in modern trade ecosystems. Conventional AML systems often rely on rule-based checks and isolated transaction analysis, which makes them less effective at identifying coordinated laundering patterns spread across connected financial networks. In practice, this leads to more false positives, more manual review, and weaker detection of sophisticated TBML activity.

This project addresses those limitations by combining heterogeneous graph learning with federated intelligence to model transaction relationships while preserving institutional privacy. By using graph attention mechanisms and collaborative training across organizations, the framework supports fraud detection without sharing sensitive transactional data directly. It also adds automated Suspicious Transaction Report (STR) generation and dashboard-based monitoring to support real-time compliance analysis.

1.4 Problem Statement

Existing AML systems rely on rules and isolated transaction analysis, which limits their ability to detect TBML and hidden financial networks. Privacy and regulatory constraints also make cross-institution collaboration difficult. A scalable and privacy-preserving framework is therefore needed to model interconnected financial data and detect suspicious activity in real time.

1.5 Objectives

The objectives of the project are

1. To develop a Federated Heterogeneous Graph Neural Network framework for real-time detection of Trade-Based Money Laundering activities.
2. To model complex transactional relationships among financial entities using heterogeneous graph representation and graph attention mechanisms.
3. To generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions to support financial investigation and compliance processes.

4. To design an interactive dashboard for visualization of fraud predictions, transaction analysis, and real-time monitoring of suspicious activities.

1.6 Brief Methodology of the project

The proposed methodology builds a privacy-preserving graph-based framework for real-time TBML detection. Financial transaction data, trade records, customer information, and entity relationships are first collected and cleaned to create structured analysis datasets. These records are then converted into a heterogeneous graph where accounts, organizations, invoices, and transactions are linked as nodes and edges.



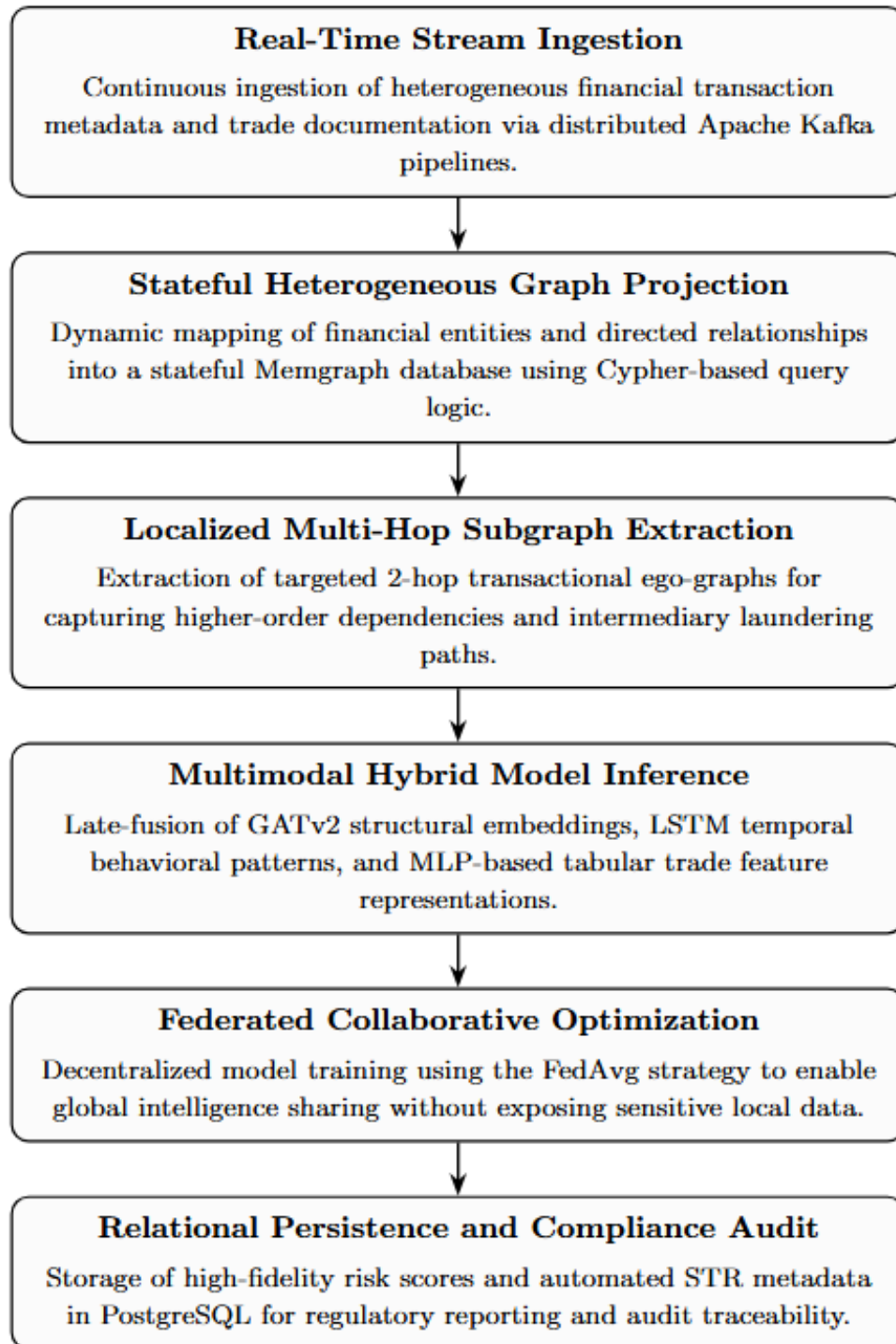


Figure 1.1: Proposed Methodology for TBML Detection

Graph attention mechanisms are used to capture hidden relationships and suspicious behavioral patterns within the financial ecosystem. Federated learning is then added so that institutions can train collaboratively without exposing sensitive local data. The detected suspicious activities are finally used to generate STRs and support real-time

monitoring through an interactive dashboard.

1.7 Assumptions and Constraints of the Work

Assumptions

- The financial transaction records, trade invoices, and customer-related information used for analysis are assumed to be sufficiently structured, consistent, and preprocessed for effective model training and evaluation.
- It is assumed that participating financial institutions maintain similar transactional formats and standardized trade-related attributes required for graph construction and suspicious activity analysis.
- The generated financial datasets are assumed to contain sufficient normal and suspicious transactional patterns necessary for learning hidden behavioral relationships associated with Trade-Based Money Laundering activities.
- It is assumed that heterogeneous graph representations can effectively model relationships among accounts, organizations, transactions, invoices, and trade entities for identifying suspicious financial behavior.

Constraints

- Limited access to real-world Trade-Based Money Laundering datasets due to institutional privacy regulations and confidentiality policies may affect the diversity of transactional data used for evaluation.
- Large-scale graph construction and real-time transaction analysis may increase computational complexity and memory requirements during fraud detection operations.
- The performance of the proposed framework depends significantly on the quality, completeness, and accuracy of the financial data available for model training and testing.
- The proposed framework is validated primarily under controlled academic and experimental conditions and may require additional optimization for large-scale industrial deployment.

1.8 Organization of the Report

This report is organized as follows.

- Chapter 2 discusses the theoretical foundations associated with Trade-Based Money Laundering detection, including Anti-Money Laundering systems, heterogeneous graph modeling, Graph Neural Networks, and federated learning methodologies relevant to the proposed framework.
- Chapter 3 illustrates the Software Requirement Specification (SRS) of the proposed system, including functional requirements, system constraints, software and hardware requirements, and operational specifications.
- Chapter 4 presents the overall system architecture and high-level design of the proposed framework, including workflow representations, component interactions, data flow mechanisms, and dashboard architecture.
- Chapter 5 elaborates the detailed system design methodology, including graph construction mechanisms, fraud-detection workflows, STR generation pipelines, and compliance-monitoring integration.
- Chapter 6 describes the implementation procedures adopted for the proposed framework, including dataset preprocessing, graph generation, backend development, federated model integration, and dashboard deployment.
- Chapter 7 evaluates the testing and validation procedures performed on the proposed system, including functional validation, fraud prediction analysis, dashboard verification, and model performance evaluation.
- Chapter 8 analyzes the experimental results and performance outcomes of the proposed framework using fraud-classification metrics, scalability observations, and system-level analytical evaluation.
- Chapter 9 summarizes the overall conclusions of the proposed work while additionally discussing the current limitations and potential future enhancements of the framework.

Chapter 2

Theory and Fundamentals of Project

CHAPTER 2

THEORY AND FUNDAMENTALS OF PROJECT

Developing an intelligent framework for Trade-Based Money Laundering detection requires a strong foundation in financial crime analysis, graph theory, and machine learning techniques. This chapter presents the fundamental concepts associated with Trade-Based Money Laundering, Graph Neural Networks, Graph Attention mechanisms, heterogeneous graph representation, and federated learning. The chapter further discusses the computational principles involved in relational transaction modeling, privacy-preserving learning, and real-time fraud analytics. These concepts collectively form the theoretical foundation for the proposed fraud detection framework.

2.1 Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) is a sophisticated form of financial crime in which illicit funds are transferred and concealed through seemingly legitimate international trade activities. Unlike conventional laundering methods that primarily depend on direct financial transactions, TBML exploits import-export systems, trade invoices, shipping documents, and cross-border commercial operations to disguise the movement of illegal funds. Due to the increasing implementation of strict Anti-Money Laundering (AML) regulations and Know Your Customer (KYC) policies, criminal organizations have increasingly shifted toward trade-oriented laundering mechanisms that are comparatively difficult to detect using traditional monitoring systems.

The complexity of modern global trade ecosystems, combined with the involvement of multiple financial entities and transactional relationships, has made TBML detection a major challenge for financial institutions and regulatory agencies. Consequently, intelligent analytical techniques capable of identifying hidden transactional dependencies and suspicious financial behavior have become increasingly important in modern Anti-Money Laundering systems.

2.1.1 Introduction to TBML

Trade-Based Money Laundering primarily involves the manipulation of international trade transactions to illegally transfer value between entities located across different countries. Criminal organizations often misuse trade documentation, invoice values, shipment quantities, and commercial agreements to move illicit funds while making the transac-

tions appear legitimate within global financial systems. Since these laundering activities are embedded within regular import-export operations, identifying suspicious behavior becomes highly challenging for financial institutions and regulatory agencies.

In many TBML operations, colluding buyers and sellers intentionally manipulate the declared value of traded goods to transfer illegal financial assets across borders. Techniques such as over invoicing, under invoicing, phantom shipments, and multiple invoicing are commonly used to conceal the movement of illicit proceeds. Such laundering mechanisms exploit the complexity and high transactional volume of international trade systems, making traditional rule-based Anti-Money Laundering approaches comparatively ineffective in detecting hidden financial relationships.

Figure 2.1 illustrates a typical Trade-Based Money Laundering workflow involving collusion between exporters and importers across different jurisdictions [14].



Figure 2.1: Trade-Based Money Laundering through trade misinvoicing and open-account transactions

2.1.2 Common Techniques Used in TBML

Trade-Based Money Laundering (TBML) involves manipulating trade documents, shipment details, and financial transactions to move illicit funds across borders while

making them appear legitimate. Criminal organizations exploit the complexity of international trade systems to avoid detection and conceal illegal financial activities.

Over Invoicing

Over invoicing occurs when the declared value of goods is intentionally set higher than the actual market price. This allows excess money to be transferred under the appearance of legitimate trade payments.

Under Invoicing

Under invoicing involves declaring goods at a lower value than their actual market price. This technique is commonly used to hide profits, evade taxes, and move funds secretly across borders.

Multiple Invoicing

Multiple invoicing refers to using the same trade invoice for several financial transactions related to a single shipment. This helps criminals transfer illicit funds through different banking channels.

Phantom Shipments

Phantom shipments occur when payments are made for goods that are never actually shipped or delivered. Fake trade and shipping documents are used to create the appearance of legitimate trade activity.

Misrepresentation of Goods

Misrepresentation of goods involves providing false information about the type, quantity, or quality of traded products in invoices and trade documents to manipulate trade values and conceal suspicious activities.

2.1.3 Challenges in Detecting TBML

Detecting TBML is difficult because modern trade systems are highly complex and interconnected. Unlike traditional financial fraud, TBML activities are often hidden within legitimate business transactions, making suspicious behavior hard to identify. Criminal networks use shell companies, intermediaries, multiple jurisdictions, and layered transactions to move illicit funds while appearing as normal trade operations.

Traditional AML systems mainly rely on rule-based monitoring and isolated transaction analysis. Although these methods can detect predefined suspicious patterns, they of-

ten fail to identify hidden relationships between entities involved in large trade networks. In addition, the growing volume of cross-border transactions and evolving laundering techniques make detection even more challenging. Privacy and regulatory restrictions also limit the sharing of sensitive financial data between institutions, creating the need for scalable and privacy-preserving TBML detection frameworks.

2.2 Algorithms Used

This section presents the major algorithms and deep learning techniques used in the proposed system for traffic prediction and autonomous vehicle navigation. The project combines graph-based learning and sequential modeling approaches to effectively analyze complex traffic patterns and temporal dependencies. Graph Attention Network v2 (GATv2) is utilized to capture spatial relationships between connected road networks through dynamic attention mechanisms, while Long Short-Term Memory (LSTM) is employed to learn temporal traffic variations and long-term sequential dependencies. Together, these algorithms improve traffic prediction accuracy, congestion analysis, and intelligent decision-making within the proposed framework.

2.2.1 Graph Attention Network v2 (GATv2)

Graph Attention Network v2 (GATv2) is a Graph Neural Network (GNN) architecture that uses a dynamic attention mechanism to learn relationships between connected nodes in graph data. It improves feature aggregation by automatically identifying the importance of neighboring nodes. GATv2 is widely used in applications such as traffic prediction, recommendation systems, and autonomous vehicle navigation.

Introduction

Graph Attention Network v2 (GATv2) is an advanced Graph Neural Network (GNN) architecture designed for learning on graph-structured data using a dynamic attention mechanism. It improves upon the original GAT model by overcoming the limitation of static attention and enabling better learning of relationships between connected nodes. GATv2 automatically identifies the importance of neighboring nodes during feature aggregation, making it highly effective for applications such as traffic prediction, recommendation systems, social network analysis, fraud detection, and autonomous vehicle navigation.

Fundamental Concept of GATv2

The primary objective of GATv2 is to generate node embeddings by aggregating information from neighboring nodes using a dynamic attention mechanism. For each node, the model learns which neighboring nodes are important and assigns suitable attention weights before combining their features.

Architecture and Working of GATv2

The overall architecture and workflow of GATv2 are shown in Figure 1.1.

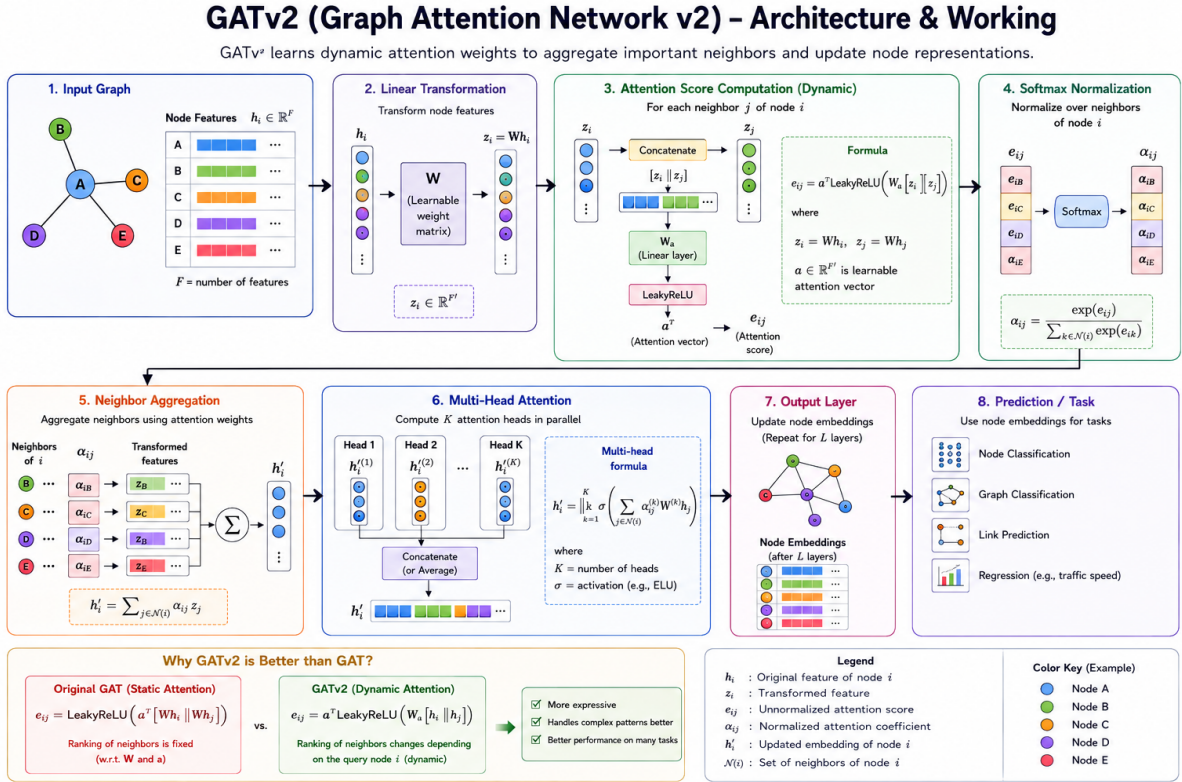


Figure 2.2: GATv2 Architecture and Working

The architecture of GATv2 consists of graph input processing, feature transformation, attention computation, normalization, neighbor aggregation, multi-head attention, and prediction generation.

Step 1: Input Graph: The process begins with a graph containing nodes, edges, and node feature vectors. Each node feature vector is represented as:

$$h_i \in \mathbb{R}^F$$

where h_i denotes the feature vector of node i and F represents the number of input features. In traffic prediction systems, nodes may represent road intersections while edges represent road connectivity.

Step 2: Linear Transformation: The input node features are transformed using a learnable weight matrix:

$$z_i = Wh_i \quad (2.1)$$

where W is the learnable weight matrix and z_i is the transformed feature representation. This transformation improves feature learning before attention computation.

Step 3: Attention Score Computation: For every connected node pair (i, j) , the attention score is computed as:

$$e_{ij} = a^T \text{LeakyReLU}(W[h_i || h_j]) \quad (2.2)$$

where a is the learnable attention vector and e_{ij} represents the attention score between neighboring nodes.

Step 4: Softmax Normalization: The attention scores are normalized using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (2.3)$$

where α_{ij} denotes the normalized attention coefficient and $\mathcal{N}(i)$ represents the neighboring nodes of node i .

Step 5: Neighbor Aggregation: The neighboring node features are aggregated using weighted summation:

$$h'_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j \quad (2.4)$$

where h'_i is the updated node representation.

Step 6: Multi-Head Attention: GATv2 employs multiple attention heads operating in parallel:

$$h'_i = \big\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} h_j \right) \quad (2.5)$$

where K represents the number of attention heads and $\parallel$ denotes concatenation. Multi-head attention improves representation learning and robustness.

Step 7: Output Layer and Prediction: The final node embeddings are passed to the output layer for tasks such as node classification, graph classification, link prediction, and regression.

Advantages of GATv2

GATv2 provides a dynamic attention mechanism that improves feature aggregation and representation learning compared to the original GAT architecture. It automatically identifies important neighboring nodes, performs effectively on irregular graph data, and improves robustness through multi-head attention, making it highly suitable for traffic prediction and autonomous navigation applications.

Limitations of GATv2

Despite its advantages, GATv2 requires high computational and memory resources due to attention computations on large graphs. Training time increases with graph size, and deep GATv2 architectures may suffer from over-smoothing and scalability issues for extremely large datasets.

GATv2 vs GAT

Graph Attention Network v2 (GATv2) was introduced as an improvement over the original Graph Attention Network (GAT) to overcome the limitation of static attention mechanisms. While both architectures use attention-based aggregation for graph representation learning, GATv2 provides a more expressive and adaptive attention mechanism.

In the original GAT architecture, the attention mechanism is defined as:

$$e_{ij} = \text{LeakyReLU} \left(a^T [W h_i \parallel W h_j] \right) \quad (2.6)$$

In GATv2, the order of operations is modified as:

$$e_{ij} = a^T \text{LeakyReLU}(W[h_i \| h_j]) \quad (2.7)$$

This modification allows the attention scores to dynamically depend on both source and neighboring nodes, improving feature interaction learning and representation capability.

Table 2.1: Comparison Between GAT and GATv2

Feature	GAT	GATv2
Attention Type	Static Attention	Dynamic Attention
Neighbor Ranking	Fixed	Context-dependent
Expressiveness	Limited	More expressive
Flexibility	Lower	Higher
Performance	Good	Improved on many graph tasks

GATv2 is preferred over GAT because its dynamic attention mechanism enables better learning of complex graph relationships, resulting in improved feature aggregation, representation learning, and prediction accuracy in graph-based applications.

2.2.2 Long Short-Term Memory (LSTM)

Introduction to LSTM

Long Short-Term Memory (LSTM) is an advanced Recurrent Neural Network (RNN) architecture designed to learn long-term dependencies in sequential data. It was introduced to overcome the vanishing gradient problem faced by traditional RNNs through the use of memory cells and gating mechanisms. LSTM can selectively remember, update, and forget information, making it highly effective for applications such as traffic prediction, speech recognition, natural language processing, and time-series forecasting.

Fundamental Concept of LSTM

The primary objective of LSTM is to preserve important sequential information over long time intervals using memory cells and gating mechanisms. The architecture controls information flow through input, forget, and output gates, enabling efficient learning of temporal dependencies.

Architecture and Working of LSTM

The overall workflow of LSTM consists of gate computation, memory cell update, and hidden state generation.

Input Gate, Forget Gate, and Output Gate

At each time step, the current input and previous hidden state are used to compute the input, forget, and output gates using sigmoid activation functions. These gates regulate information flow into and out of the memory cell.

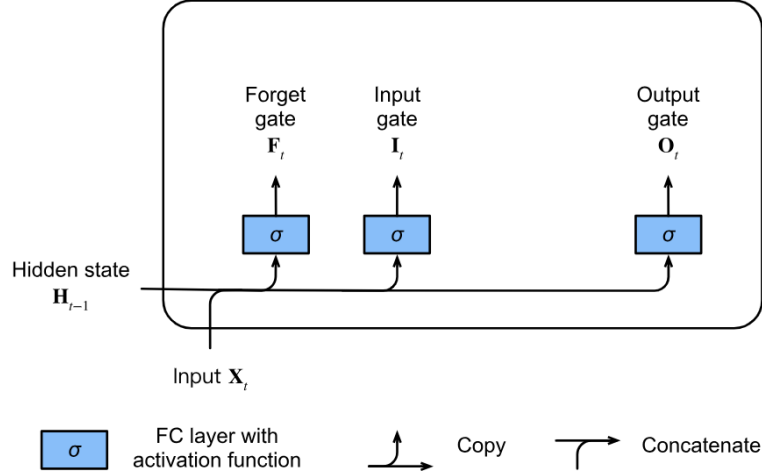


Figure 2.3: Computing the input gate, forget gate, and output gate in an LSTM model

The gates are mathematically represented as:

$$\begin{aligned} \mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i) \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f) \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o) \end{aligned} \quad (2.8)$$

where $\mathbf{I}_t$, $\mathbf{F}_t$, and $\mathbf{O}_t$ represent the input, forget, and output gates respectively, while $\mathbf{X}_t$ and $\mathbf{H}_{t-1}$ denote the current input and previous hidden state.

Input Node

The candidate memory information is computed using the **tanh** activation function as:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c) \quad (2.9)$$

where $\tilde{\mathbf{C}}_t$ represents the candidate memory state.

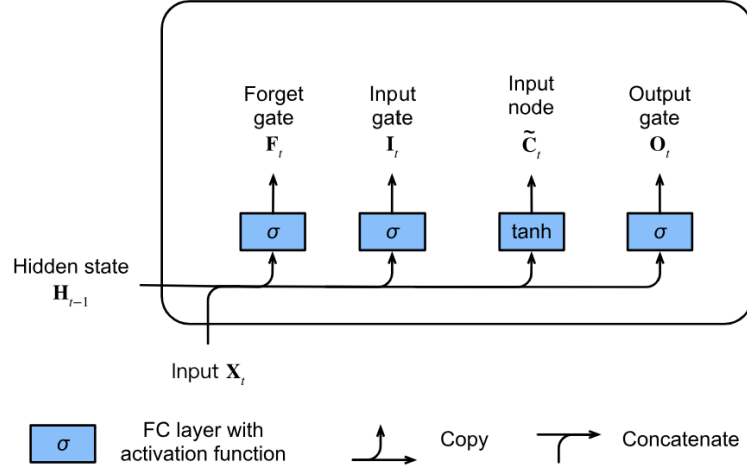


Figure 2.4: Computing the input node in an LSTM model

Memory Cell Internal State

The memory cell state is updated using the forget gate and input gate as:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \quad (2.10)$$

where $\mathbf{C}_t$ represents the updated memory cell state and $\odot$ denotes element-wise multiplication. This mechanism helps preserve important long-term information while discarding irrelevant data.

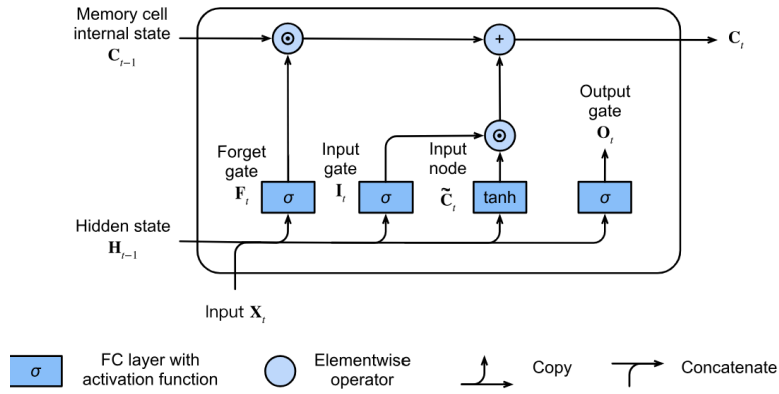


Figure 2.5: Computing the memory cell internal state in an LSTM model

Hidden State

The hidden state output is computed as:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \quad (2.11)$$

where $\mathbf{H}_t$ represents the hidden state at time step t . The output gate controls how much information from the memory cell is passed to the next layer.

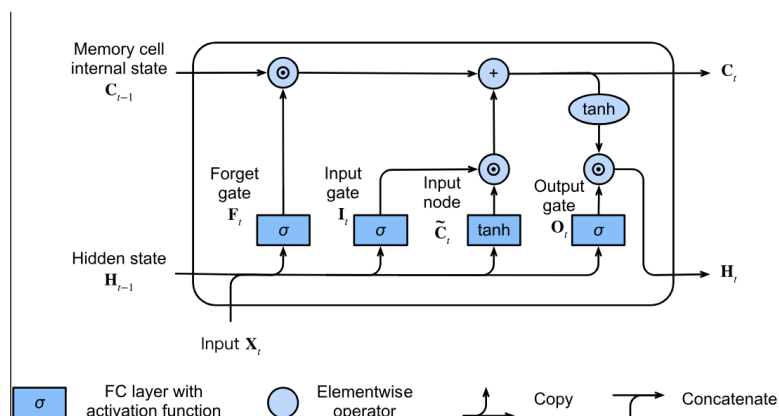


Figure 2.6: Computing the hidden state in an LSTM model

Advantages of LSTM

LSTM effectively solves the vanishing gradient problem and learns long-term sequential dependencies using memory cells and gating mechanisms. It performs efficiently on sequential and time-series data and is widely used in applications such as traffic prediction, speech recognition, machine translation, and forecasting.

Limitations of LSTM

Despite its advantages, LSTM is computationally expensive due to multiple gates and memory operations. Training requires higher computational resources and longer execution time, and performance may decrease for extremely long sequences or insufficient training data.

Source: Adapted from “Dive into Deep Learning, Long Short-Term Memory (LSTM).”

2.3 Federated Learning

Federated Learning (FL) is a decentralized machine learning approach where multiple devices or organizations collaboratively train a shared model without exchanging raw data. Each client performs local training on its own dataset and shares only model parameters with a central server, enabling privacy-preserving collaborative learning across distributed environments.

2.3.1 Introduction

Traditional machine learning systems rely on centralized training, where data from multiple users or organizations is collected and stored on a central server for model development. Although effective, this approach raises significant concerns related to data privacy, security, and regulatory compliance. Federated Learning addresses these challenges by enabling collaborative model training without sharing confidential or sensitive data.

In Trade-Based Money Laundering (TBML) detection systems, organizations such as banks and financial institutions can locally train models using transaction records and suspicious trade patterns while sharing only model parameters with a central server. Introduced by Google in 2016 for Gboard applications, Federated Learning has emerged as an important privacy-preserving approach for distributed machine learning systems .

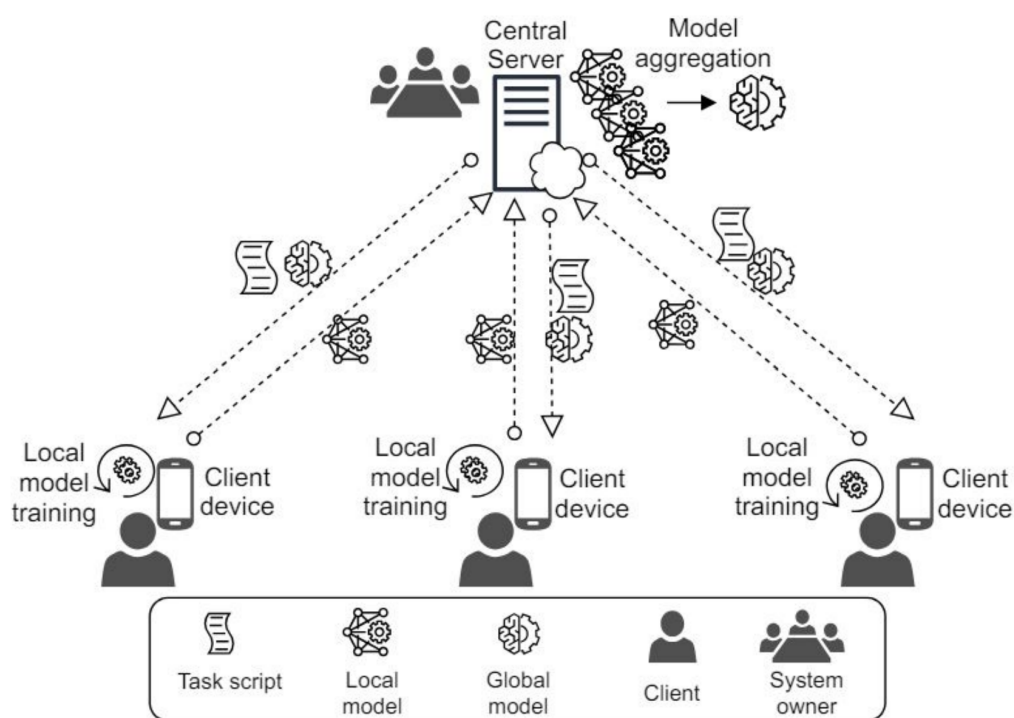


Figure 2.7: General Outlook of Federated Learning

Federated Learning also reduces communication overhead because only compact model updates are exchanged instead of complete datasets. This makes FL suitable for large-scale distributed environments involving mobile devices, IoT systems, edge devices, and financial institutions [15].

2.3.2 Working of Federated Learning

Federated Learning operates through iterative client-server interactions known as federated rounds. During each round, the central server distributes the current global model to selected clients, local training is performed independently on local datasets, and the updated model parameters are aggregated to generate an improved global model.

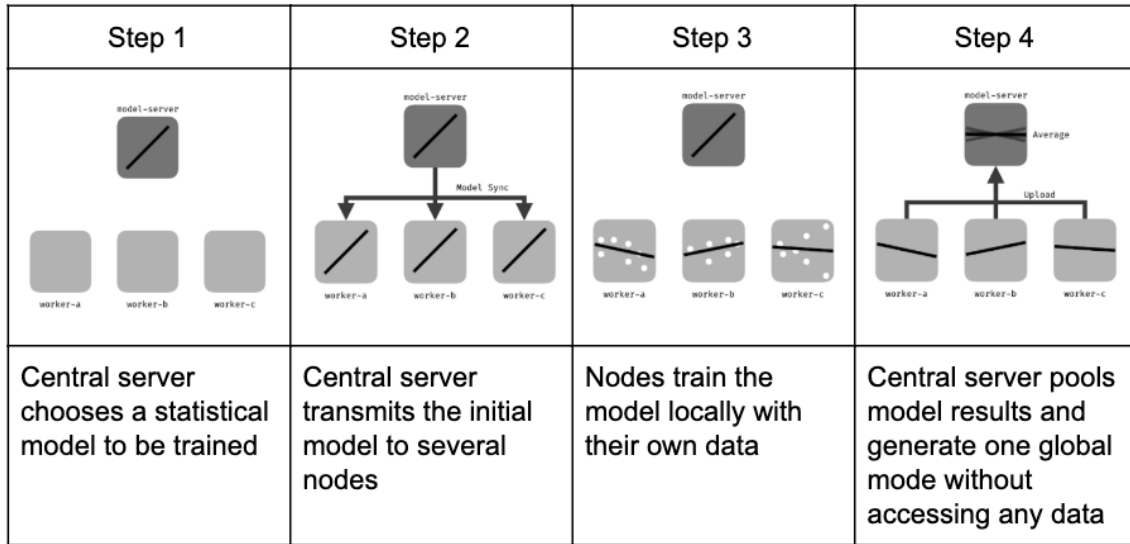


Figure 2.8: Federated Learning General Process in Central Orchestrator Setup

Step 1: Initialization An initial machine learning model is selected and initialized at the central server. Client devices or institutional servers are then prepared to participate in local training.

Step 2: Client Selection During each federated round, only a subset of clients is selected for training. These clients receive the latest global model parameters from the server.

Step 3: Local Training Each selected client trains the received model locally using its private dataset without sharing raw transactional or customer information.

Step 4: Reporting and Aggregation After local training, clients send only model parameters or gradients to the aggregation server. The server combines these updates using aggregation algorithms such as Federated Averaging (FedAvg).

The Federated Averaging process is represented as:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k^{(t+1)} \quad (2.12)$$

where w_{t+1} represents the updated global model, $w_k^{(t+1)}$ denotes the local model of client k , n_k represents the number of training samples at client k , and n denotes the total number of training samples across all clients.

Step 5: Termination The federated training process continues iteratively until the desired accuracy, convergence criteria, or maximum communication rounds are achieved.

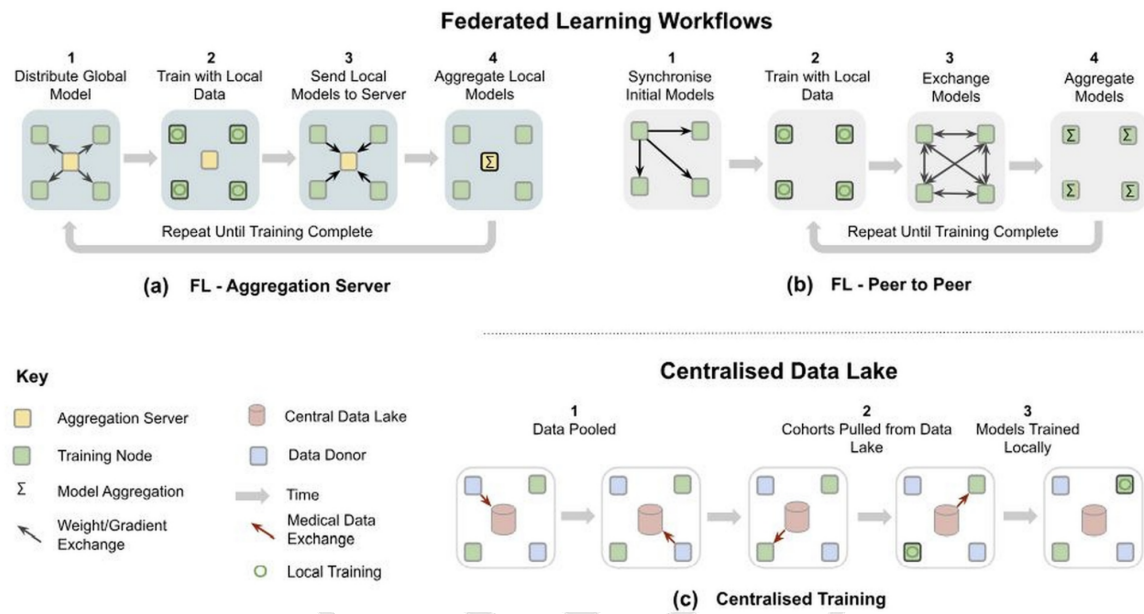


Figure 2.9: Federated Learning Workflows showing (a) FL-Aggregation Server, (b) FL-Peer-to-Peer, and (c) Centralized Training architectures.

2.3.3 Advantages of Federated Learning

Federated Learning improves data privacy and security by ensuring that raw data remains within local devices or organizations. It supports regulatory compliance, enables collaborative intelligence without direct data sharing, reduces communication overhead, and improves model generalization using diverse distributed datasets. FL is also scalable and supports real-time learning on edge devices and organizational servers.

2.3.4 Limitations of Federated Learning

Despite its advantages, Federated Learning introduces challenges such as communication delays, high computational requirements, non-i.i.d. data distributions, client failures,

synchronization issues, and security threats such as model poisoning and backdoor attacks. Variations in client hardware capabilities and unstable network connections can also affect training efficiency and global model convergence.

2.4 User Interface

Modern financial monitoring platforms increasingly rely on scalable web application technologies to support real-time transaction analysis, fraud monitoring, and interactive visualization of financial activities. The rapid growth of digital banking and online financial services has significantly increased the demand for responsive, secure, and data-driven monitoring systems capable of handling large volumes of transactional information efficiently. Consequently, modern Anti-Money Laundering systems employ component-based web architectures and client-server communication models to provide efficient user interaction and real-time analytical capabilities.

Advanced frontend frameworks and backend service architectures play a critical role in enabling modular application development, dynamic rendering, secure authentication, and seamless integration with fraud detection engines. Such technologies support the visualization of suspicious transactional activities, alert generation, role-based access control, and automated compliance workflows within financial institutions. The integration of scalable user interface technologies with intelligent analytical systems further enhances operational efficiency and supports real-time financial crime investigation processes.

2.4.1 Modern Web Application Frameworks

Modern web application frameworks provide the architectural foundation required for developing scalable, responsive, and interactive software systems. Traditional web applications primarily relied on static page rendering and tightly coupled server-side architectures, which limited dynamic user interaction and real-time data processing capabilities. In contrast, modern frameworks adopt component-based and asynchronous rendering approaches that improve application scalability, maintainability, and user experience within large-scale distributed systems.

Frontend frameworks such as React.js enable the development of reusable and modular user interface components capable of dynamically updating application states without requiring complete page reloads. Such frameworks employ virtual Document Object Model (DOM) mechanisms and state-driven rendering principles to efficiently manage user in-

teractions and real-time data visualization. This architectural approach significantly improves responsiveness and enables scalable dashboard development for data-intensive applications such as financial monitoring systems.

Server-side rendering and hybrid rendering frameworks such as Next.js further enhance modern web application performance by combining client-side interactivity with optimized server-side content delivery. These frameworks support efficient routing, dynamic data fetching, and improved application scalability while reducing rendering overhead for large-scale systems. Consequently, modern web application frameworks have become an essential foundation for building secure, scalable, and real-time financial monitoring platforms.

2.4.2 Component Lifecycle and Reactivity

Modern frontend frameworks such as React and Next.js utilize reactive rendering mechanisms to synchronize user interfaces with continuously changing application states and backend responses. Within the proposed TBML detection framework, this reactive architecture enables efficient visualization of transaction analytics, fraud alerts, compliance activities, and real-time monitoring information without requiring complete interface reloading. The component-based design additionally improves scalability, maintainability, and responsiveness within large-scale financial monitoring environments.

The lifecycle of application components generally involves rendering, state initialization, reactive updates, synchronization, and resource cleanup operations during execution. The major lifecycle stages associated with the proposed dashboard architecture are summarized as follows:

Server-Side Rendering: Initially, dashboard components are rendered within the server environment where transaction summaries, analytical data, and monitoring information are processed before delivery to the client browser. This approach improves initial loading performance and reduces unnecessary client-side computational overhead.

Hydration and Client Initialization: After the interface is loaded within the browser, React hydration enables interactive client-side functionalities including event handling, asynchronous API communication, local state initialization, and real-time dashboard services.

Reactive State Updates: As transaction states, fraud alerts, and analytical outputs

change, React selectively updates only the affected interface components using Virtual DOM reconciliation mechanisms instead of re-rendering the complete dashboard.

Dynamic Synchronization: The framework continuously synchronizes updated transaction information, compliance alerts, and monitoring analytics with the dashboard interface to support real-time fraud visualization and analytical consistency.

Resource Cleanup and Unmounting: When dashboard components are removed from the active interface, temporary states, subscriptions, and allocated resources are released automatically to maintain application stability and efficient memory utilization.

Such lifecycle management principles are important within modern financial monitoring systems where continuous transactional updates, fraud notifications, and analytical visualizations must be processed efficiently while maintaining scalable and responsive user interaction.

2.4.3 Data Visualization

Understanding the theoretical aspects of data visualization is important for analyzing large volumes of financial transactions and interpreting suspicious transactional behavior within real-time monitoring systems. Visualization of analytical outputs enables financial institutions and compliance analysts to identify fraud patterns, transaction anomalies, risk distributions, and suspicious entity relationships more efficiently when compared with conventional tabular analysis methods. Interactive monitoring dashboards further improve situational awareness by enabling continuous observation of fraud alerts, transaction flows, and compliance activities without interrupting real-time analytical processes. This work utilizes a web-based dashboard architecture for financial monitoring, fraud visualization, and compliance-oriented analytical operations using modern frontend technologies and visualization frameworks.

Frontend Framework: Next.js is used for developing responsive and modular dashboard interfaces capable of supporting scalable financial monitoring applications. The framework provides server-side rendering, dynamic routing, and smooth integration with backend analytical services operating within the proposed TBML detection environment.

Analytical Visualization Components: Recharts is utilized to visualize transaction analytics, fraud classifications, risk-score distributions, and temporal financial trends through interactive graphs and dashboard components. These visualizations assist in-

investigators and analysts in interpreting suspicious transactional behavior more efficiently during monitoring operations.

Role-Based Dashboard Interfaces: The dashboard architecture incorporates role-based access mechanisms to provide controlled access to fraud investigation workflows, Suspicious Transaction Report generation, real-time monitoring services, and institutional compliance operations according to user responsibilities and operational privileges.

Interactive Monitoring Services: Interactive dashboard panels support continuous monitoring of suspicious transactions, fraud alerts, transaction histories, and prediction results generated by the analytical framework. The monitoring environment additionally supports filtering, notification handling, export functionalities, and compliance-oriented investigation workflows for institutional analysts and regulatory authorities.

The detailed implementation and integration of these visualization technologies within the proposed TBML detection framework are discussed further in the system implementation chapter.

2.5 Summary

In this chapter, the theoretical foundations associated with Trade-Based Money Laundering (TBML) detection were presented, including laundering methodologies, limitations of conventional Anti-Money Laundering systems, graph-based financial modeling, heterogeneous Graph Neural Networks, Graph Attention mechanisms, and federated learning concepts. The chapter additionally discussed real-time financial monitoring architectures and visualization systems used for fraud analytics and compliance operations. These concepts collectively establish the analytical and computational foundation for the proposed privacy-preserving TBML detection framework presented in the subsequent chapters.



Appendix A

Plagiarism report

APPENDIX A

PLAGIARISM REPORT

Attach the screenshot of plagiarism report front page.





Appendix B

Publication details

APPENDIX B

PUBLICATION DETAILS

Attach the screenshot of either the Acknowledgment of the publication / Proof of Acceptance / Proof of Registration.





Appendix C

Code

APPENDIX C

CODE

C.1 First Appendix

You can use `tcbllisting` for creating the code snippets. The following example illustrates how one can customize the `tcbllisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

- [1] IMTF Compliance Technologies, *When trade becomes a cover: Unpacking trade-based money laundering*, 2024. [Online]. Available: <https://imtf.com/blog/when-trade-becomes-a-cover-unpacking-trade-based-money-laundering/>
- [2] Financial Action Task Force and Egmont Group, “Trade-based money laundering: Trends and developments,” FATF, Paris, France, Tech. Rep., 2020. [Online]. Available: <https://fatf-gafi.org>
- [3] United Nations Office on Drugs and Crime, *Money laundering and Illicit financial flows overview*, 2024. [Online]. Available: <https://unodc.org>
- [4] Europol, *Criminal finances, money laundering and asset recovery*, 2024. [Online]. Available: <https://europa.eu>
- [5] J. Saenz and J. Lewer, “Quantifying illicit financial flows and trade misinvoicing in developing economies,” *Journal of Financial Crime Research*, vol. 18, no. 2, pp. 112–128, 2024.
- [6] J. Fan et al., “Deep learning approaches for anti-money laundering on mobile transactions: Review, framework, and directions,” *IEEE Internet of Things Journal*, vol. 13, pp. 10 407–10 428, 2026, ISSN: 2327-4662. DOI: 10 . 1109 / JIOT . 2026 . 3653434
- [7] A. I. Rasmus Ingemann Tuffveson Jensen, “Fighting money laundering with statistics and machine learning,” *IEEE Access*, vol. 11, pp. 8889–8903, 2023, ISSN: 2169-3536. DOI: 10 . 1109 / ACCESS . 2023 . 3239549
- [8] J. Song, S. Zhang, J. Park, Y. Gu, and G. Yu, “Dimension expansion for learning money laundering activities hidden in transaction network,” *IEEE Transactions on Computational Social Systems*, vol. 13, pp. 251–262, 2026, ISSN: 2329-924X. DOI: 10 . 1109 / TCSS . 2025 . 3599534
- [9] J. Song, S. Zhang, P. Zhang, J. Park, Y. Gu, and G. Yu, “Illicit social accounts? anti-money laundering for transactional blockchains,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 391–404, 2025, ISSN: 1556-6021. DOI: 10 . 1109 / TIFS . 2024 . 3518068

- [10] Z. Li, X. Yang, and C. Jiang, “Multi-view graph-based hierarchical representation learning for money laundering group detection,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2035–2050, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2025.3529321
- [11] A. H. B. Gezici and E. Sefer, “Pagerank-based unsupervised deep vertex representations for anti-money laundering detection,” *IEEE Access*, vol. 13, pp. 196 935–196 950, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3634197
- [12] M. R. Karim, F. Hermesen, S. A. Chala, P. D. Perthuis, and A. Mandal, “Scalable semi-supervised graph learning techniques for anti money laundering,” *IEEE Access*, vol. 12, pp. 50 012–50 029, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3383784
- [13] K. Koo, M. Park, and B. Yoon, “A suspicious financial transaction detection model using autoencoder and risk-based approach,” *IEEE Access*, vol. 12, pp. 68 926–68 939, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3399824
- [14] A. E. Iyanda, “The role of misinvoicing in the money laundering cycle,” *Journal of Anti-Corruption Law*, pp. 144–168, 2023. DOI: 10.14426/jacl.v3i1.1294
- [15] A. E. Iyanda, “Principles and components of federated learning architectures,” pp. 144–168, 2023. DOI: 10.14426/jacl.v3i1.1294